



Observatory: Fast and Scalable Systems Observability

by
Arvind Yalavarti

A Thesis submitted in partial fulfillment of the requirements for Honors
in the Department of Computer Science at Brown University

Providence, Rhode Island
December 2022

This thesis by Arvind Yalavarti is accepted in its present form by
the Department of Computer Science as satisfying the research requirement
for the awardment of Honors.

Date _____

Theophilus A. Benson, Advisor

Date _____

Nicholas DeMarinis, Reader

There is a growing adoption of microservice architectures in industry, but a key challenge with this paradigm is monitoring the state of this distributed architecture. The high volume of data generated in typical microservice deployments leads to concerns of accuracy and high latency with traditional observability frameworks where data collection and processing is centralized. In this thesis, we present Observatory, a decentralized observability framework built with extended Berkeley Packet Filter (eBPF) technology that offloads insight generation to the data plane, which traditionally only performs data collection. We design and evaluate data structures, primitives, and workflows to realize Observatory. To demonstrate the effectiveness and scalability of our framework, we implement existing state-of-the-art observability systems for problem detection and the orchestration of autoscaling in Observatory. We show, in at least one instance, that Observatory is able to create high-quality local insights up to 50% faster than with centralized observability methods.

Abstract of “Observatory: Fast and Scalable Systems Observability” by Arvind Yalavarti, Brown University, December 2022.

Acknowledgements

I would like to foremost thank Theophilus A. Benson for the opportunity to work on Observatory this past year. Secondly, this project would not have been possible without Lucas Castanheira and his invaluable support, guidance, and insights. Next, I would like to thank Nicholas DeMarinis for being the second reader on this thesis and Franco Solleza for his insightful comments and feedback. Finally, I would like to thank my family, my friends, and the faculty at Brown Computer Science for supporting me throughout my undergraduate studies. This work was funded in part by the VMware University Research Fund.

Contents

1	Introduction	1
2	Background	3
2.1	Microservices Background	3
2.2	Kubernetes Background	4
2.3	The Pillars of Observability	4
2.4	Extended Berkley Packet Filters (eBPF)	5
2.4.1	Programs	5
2.4.2	Tracepoints	5
2.4.3	Shared Storage	5
3	Motivation	6
3.1	Traditional Observability	6
3.1.1	Decentralizing Observability	7
3.2	Use Cases	8
3.2.1	Problem Diagnosis with Kahuna	8
3.2.2	Autoscaling with Sieve	8
3.2.3	Data Visibility Classes in Observability Queries	9
3.2.4	Visibility Class Definitions	9
3.2.5	Control and Data Plane Requirements	11
3.3	Observatory and eBPF	11
3.3.1	Efficiency	11
3.3.2	Flexibility	12
4	Design	13
4.1	Design Overview	13
4.2	Data Collection Pipeline	14

4.2.1	Pod Discovery and Dynamic Configuration	15
4.2.2	Histogram Data Structures	15
4.2.3	Data Streaming, Ingestion, and Processing	16
4.3	eBPF in the Data Plane	17
5	Observatory Queries	19
5.1	Query Language Overview	19
5.2	Language Constructs	20
6	Prototype & Setup	23
6.1	Prototype	23
6.2	Evaluation Setup	23
6.3	Use Cases	23
6.3.1	Problem Diagnosis with Kahuna	24
6.3.2	Autoscaling with Sieve	24
7	Evaluation	27
7.1	Use Cases	27
7.1.1	Kahuna Problem Detection Results	27
7.1.2	Sieve Autoscaling Results	29
7.2	eBPF-based Data Structures	30
7.3	Probe Latency	31
7.3.1	Critical Path Latency	31
7.3.2	Request Latency	31
7.4	Control and Data Plane Scalability	32
8	Related Works	35
9	Future Work	37
10	Conclusion	38
	Bibliography	39

Chapter 1

Introduction

Microservice architectures are becoming widely adopted by the industry, providing a wide range of benefits over the traditional, monolithic architectures, some of which include greater fault tolerance, decentralized ownership and data management, and more granular releases [24]. However, the increased complexity of microservice architectures introduces new challenges in terms of managing infrastructure, performance debugging, and ensuring availability [9, 34, 54].

One of the key challenges in adopting this paradigm is observability, or understanding the internal state of these complex, highly distributed, and dynamically changing systems. From control theory, observability in distributed systems is the ability to understand *why* events occurred in order to perform performance debugging or resource management [42]. This differs from *monitoring*, which aims to collect telemetry data from a cluster of nodes. Observability, on the other hand, uses this data to understand how different components of a system interact and to gain greater visibility into why the system is behaving a certain way.

Traditional approaches to observability in microservices face two main issues: (i) the high volume of telemetry data creates scale problems for current observability frameworks [27], which cope with scale by using *sampling*, which hinders accuracy [10]. Moreover, (ii) in current observability frameworks, hosts generate and export telemetry data (metrics) in a *centralized* manner: Client-side agents export data to a central entity for processing [41], which incurs high latency. These approaches of both sampling and centralizing data create bottlenecks in our ability to transform telemetry data into high quality insights, *i.e.*, low-latency and highly accurate insights, which are critical to several observability use cases [48, 49].

In this thesis, we show that a centralized approach to observability in microservices is not suited for use cases which require high quality insights. We propose Observatory, a decentralized observability framework that offloads insight generation to the data plane, which traditionally only

performs data collection. Observatory supports *flexible insight generation* that supports creating insights both locally and globally. The key enabling technology for our framework is extended Berkeley Packet Filters (eBPF) [2], which allows us to dynamically modify metric collection at the data plane. To address challenges with sampling and speed, we introduce custom eBPF data structures that enable Observatory to operate efficiently and scalably.

While related works [28, 53] address a subset of the challenges described with observability frameworks, our framework allows observability queries to be dynamically decomposed and offloaded to an infrastructure. As case studies, we implement existing observability frameworks for problem diagnosis (Kahuna [48]) and orchestration of autoscaling (Sieve [49]) to demonstrate the effectiveness and flexibility of Observatory.

We present the following key contributions:

1. We introduce Observatory, a decentralized observability framework with flexible insight generation, and design data structures, primitives, and workflows to realize Observatory.
2. We design a high-level *query language* for users to specify how to generate insights both locally and globally.
3. We demonstrate the flexibility and efficiency of our framework by implementing existing observability systems in Observatory and comparing against state of the art observability frameworks [41].

The remainder of this thesis is structured as follows. Chapter 2 discusses background about microservices. Chapter 3 motivates our work on observability. Chapters 4 and 5 detail the design and architecture of Observatory and introduce Observatory’s Query Language. Chapter 6 presents our prototype and evaluation setup, and Chapter 7 presents our evaluation. Chapters 8 and 9 discuss related and future work, and Chapter 10 serves as a conclusion for this thesis.

Chapter 2

Background

2.1 Microservices Background

Traditional applications were built as *monolithic*, single unit applications. To address increasing or decreasing load, monoliths can horizontally scale by deploying more instances of the entire application. Some issues with the monolithic architecture style include development cycles of distinct components being tied together, having to rebuild and redeploy the entire application at each cycle (often with entirely unchanged components), and scaling the entire application (as opposed to just the components that are facing changes in load) [30].

The microservice architectural style builds applications as a suite of independent services. Each service can be designed, built, tested, deployed, managed, and scaled independently of other services. Services communicate with one another with well-defined APIs and use HTTP or remote procedure calls (RPCs) when necessary. This leads to a heterogeneous environment of services, often written in different languages and managed by different teams.

Many companies in industry have adopted microservice architectures (Google [25], Uber [47], Netflix [29], Spotify [54]). These architectures provide a wide range of benefits [24] over the monolithic style which include greater fault tolerance and scalability, decentralized ownership and data management, isolation of resources, failures, and environments between services, and faster application releases and feature delivery. However, the increased complexity, scale, and distributed nature of microservices architectures brings forth new challenges and active research in resource management [14, 51], testing and reliability [7], performance debugging [21, 44, 48], and distributed tracing [20, 45].

2.2 Kubernetes Background

In a microservice deployment, different components are known as *services*. The *service graph* is a directed acyclic graph that captures the flow of requests between services and is widely used to understand the relationships between services. Deployments typically run on *containers* that provide simpler management functionality. Containers are designed to virtualize the operating system and provide the ability to run multiple environments on a single host by sharing the host operating system kernel. *Docker* [1], a variant of containers, increased the popularity of containerized deployments, and technologies such as *Kubernetes* [12] provide container orchestration, the ability to spin up and down hundreds of containers running different services on demand [15].

With the hundreds to thousands of services running in a typical microservice deployment at large companies [31], each collecting and exporting its own set of data, there is a vast amount of data generated. The ability to cope with this vast amount of data is critical for Observatory and will guide the design of our solution throughout this thesis.

2.3 The Pillars of Observability

The three *pillars* of observability in distributed systems are *logs*, files that record errors, warnings, events, and more during the lifetime of the application, *metrics*, measurements taken that reflect the health or performance the application or the system, and *distributed traces*, paths that application requests follow as they move from service to service [46]. Active research in observability systems that operate on logs [36, 50], metrics [11, 48, 49, 56], and distributed traces [18, 22, 44, 45] address either collecting, exporting, and storing data at scale or generating actionable insights from collected telemetry data.

While logs and distributed traces are both useful in providing visibility into a distributed system, our work with Observatory focuses on metrics for the following reasons:

1. Logs and distributed traces require instrumenting the target application [40, 46]. System metrics can be collected tangentially to any running application and does not require modifying or instrumenting the target application.
2. Metrics are more suitable to mathematical and statistical operations after being collected, where as logs are generally unstructured and must be parsed either at the client or the server before these operations can be performed [46].

2.4 Extended Berkley Packet Filters (eBPF)

eBPF is a Linux kernel technology that allows developers to run sandboxed programs in the operating system’s kernel and does not require changes to the kernel source code [2]. The operating system is traditionally an ideal location to gather system metrics and implement observability solutions due to the kernel’s privileged abilities and visibility into the system’s state. For instance, eBPF programs can add simple, lightweight [2] monitoring to common syscalls such as `malloc` or `execv` or counters for system events such as virtual file system reads and writes. This exposes interesting, app-agnostic performance metrics. Many companies in industry have adopted eBPF in their infrastructure, *e.g.*, Facebook has incorporated eBPF in their network load balancing [19]. The following key properties of eBPF guide our design of Observatory.

2.4.1 Programs

eBPF programs are compiled from a source language (*e.g.*, a pseudo-C language) into eBPF bytecode. The bytecode is loaded into the kernel and run through a static *verifier*. The verifier ensures safety of the BPF program, with properties including running to completion without unbounded loops, accessing valid memory, meeting size requirements, and having a finite complexity [33]. We discuss in Section 4.3 how we leverage the flexibility of eBPF programs in our data collection pipeline.

2.4.2 Tracepoints

eBPF programs are run directly in the Linux kernel [2]. This allows us to instrument the host’s kernel by placing metric collection logic at tailored *tracepoints* or *kprobes*. A tracepoint or *kprobe* is a set conditions that are met in the kernel while it performs certain tasks. When one of these are met, we can extract information about the process and the state of the kernel. A kernel-space program is written by users and hooks on to the tracepoint. When the tracepoint is met, the user-defined code is executed.

2.4.3 Shared Storage

An eBPF map is a special data structure that provides a generic storage mechanism for sharing data between kernel-space and user-space. Kernel-space programs can read and atomically write data into eBPF maps, and user-space programs can read, update, and delete map elements through syscalls. eBPF maps can be used to collect observed data and maintain state between invocations of the kernel-space eBPF program.

Chapter 3

Motivation

In this section, we discuss the motivation for designing a dynamic, decentralized observability framework with flexible insight generation.

3.1 Traditional Observability

We first discuss traditional observability frameworks and their limitations as motivation for decoupling data collection from insight generation and designing specialized data structures for collecting metrics.

In current observability frameworks, data is *centralized*. Essentially, we observe the following characteristics of data collection pipelines, illustrated in Figure 3.2:

1. **Data collection:** Observability agents on hosts acquire raw system or application metrics and perform some form of sampling, aggregating, or filtering to reduce the volume of data.
2. **Data ingestion:** Data from hosts is exported to a *global* collector (*e.g.* Prometheus [41]) with a centralized data processing pipeline (*e.g.*, Apache Kafka [26]).
3. **Insight generation:** Finally, at the global entity, the data can be processed and analyzed *post-hoc* to generate insights.

This paradigm creates a bottleneck in our ability to generate fast, accurate insights. We first examine data sampling at the data collection layer. Sampling is often employed to account for the high volume of data generated across the deployment. For instance, Netflix reported receiving over 1.2 billion metrics at Atlas (their telemetry platform) during one failover exercise [35]. However, data sampling could lead to a loss in accuracy [10]. We illustrate the effects of sampling in Figure

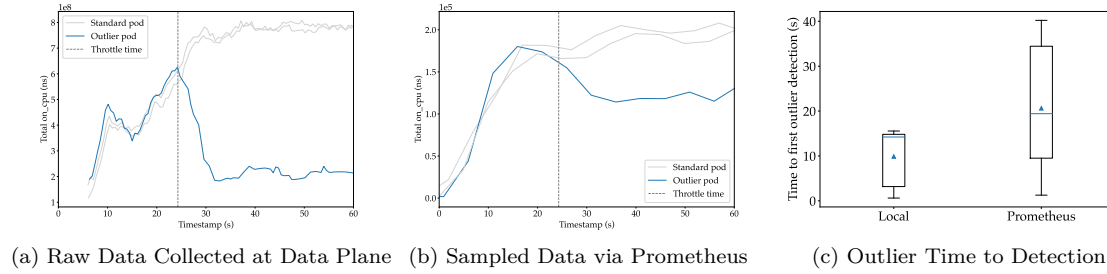


Figure 3.1: Effects of Sampling

3.1, where we compare the raw data collected by Observatory at the data plane and the data available to the control plane via Prometheus [41] in a problem diagnosis application. Prometheus is a commonly used monitoring system for timeseries data that scrapes client endpoints for metrics. However, Prometheus only pulls data adhering to a fixed scraping interval. During each scrape, the *current* values of all metrics are read and added to their timeseries vectors at the central Prometheus instance, meaning that incremental changes between scrapes are lost and the resulting data is sampled. In Figure 3.1b, we see the data read from Prometheus by the controller is too granular for this application compared to that of Figure 3.1a: the fewer data points reduces the predictive power of the outlier detection algorithm. Other observability systems also benefit from high-fidelity data and rely on fast insights. For instance, Seer [22] attempts to predict upcoming *Quality of Service* (QoS) violations though deep learning on distributed tracing data and then utilizes low-level hardware monitoring for problem diagnosis. The slower data is collected and processed, the slower the data is analyzed and insights are generated, and the less actionable and effective these systems are. This experiment motivates our need for a new approach to observability that prevents the need for sampling.

3.1.1 Decentralizing Observability

Decentralized data collection pipelines can be shown to avoid sampling and high latency in specific use cases. We illustrate this in Figure 3.1c where we compare time to detection of a faulty microservice pod using data from *local insights* (insights generated locally at the data plane) with data from a centralized Prometheus. We see that time to detection is higher when generating insights globally. Specifically for this use case, the granular curve has increased time to detection over that of Prometheus by 52%. Generally, higher granularity and lower latencies offered at the node-level can help the performance of observability systems [48, 22, 49].

We next elaborate on a few use cases of interest to Observatory that motivate our need for

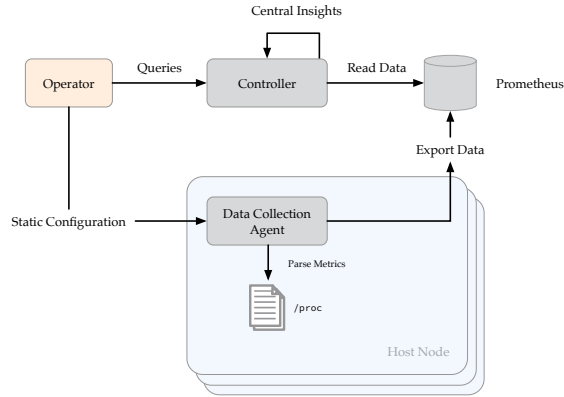


Figure 3.2: Centralized Data Collection Pipeline Architecture

a generic, customizable approach to decentralized observability, where insights can be generated alongside data collection.

3.2 Use Cases

Our goal in this section is to elaborate on a few use cases of interest to Observatory to motivate the requirements for primitives, queries, and the overall design of Observatory.

3.2.1 Problem Diagnosis with Kahuna

Kahuna [48] is a problem diagnosis approach that relies on *peer-similarity*. Nodes are claimed to exhibit similar behavior without problems, whereas nodes that behave significantly different than its peers are likely to be sources of problems. Each node feeds a vector of metrics to the problem diagnosis algorithm, which then performs a pairwise comparison between every pair of nodes. If any node's vector is further away from a majority of vectors of other nodes, that node is declared to be an outlier. Due to the pairwise comparison, detecting outliers requires knowledge of the state on multiple (but not all) nodes in the cluster.

3.2.2 Autoscaling with Sieve

Sieve [49] is a platform to generate actionable insights from a distributed system. While Sieve itself was shown to be effective in supporting a wide range of workflows, our work in Observatory focuses on a specific workflow: orchestration of autoscaling. Given a set of monitored metrics in a distributed system, Sieve's autoscaling engine selects some metric m to use for autoscaling

decisions. Based on the value of this metric on nodes in a cluster, Sieve decides whether or not the node is experiencing a surge in load and, if so, horizontally scales the affected node. All insights on whether a node is experiencing a surge in load can be performed locally using metric data collected on an individual node. Specifically, we can compare current and historic metric data on a node to determine whether there is a surge in load.

3.2.3 Data Visibility Classes in Observability Queries

We next specify query decomposition primitives based on the visibility requirements of different analysis techniques. A *query* in Observatory is some introspection into a microservice deployment. For instance, an SRE could query our system for *"the P90 memory usage across all nodes in a cluster"*. However, queries are not limited to individual or aggregated metrics, *e.g.*, an SRE could also query for *"any outlier pods in service X based on memory usage"*.

3.2.4 Visibility Class Definitions

We survey a broad range of techniques and algorithms used in existing observability systems to identify common paradigms and properties needed in data collection and insight generation to motivate the design, interface, primitives, and workflows of Observatory. Table 3.1 displays a set of existing tools, techniques, and algorithms that could conceivably be implemented in the Observatory framework as queries. From this survey, we observe that different queries require different levels of *visibility* into the collected data, and we can define three classes of visibility:

- **Local Insights:** Insights can be generated *fully locally*, that is, on the hosts themselves. The data collected on each individual node is sufficient to generate insights independently of other nodes in a cluster.

For example, an SRE could decide to autoscale with Sieve based on the P90 memory of all services in a cluster, and the procedure defined in Section 3.2.2 can be performed locally using data available on each individual node.

- **Group Data Sharing:** Many techniques employ some notion of a *peer-group* or *cohort* of nodes in a cluster that share some property, *e.g.* the assigned MapReduce job (Kahuna [48]) in a cluster. In this class, insights can *still be generated locally*, but nodes need to be aware of the state on other nodes in their peer-group.

For example, an SRE could decide to query for outlier pods with Kahuna based on the cumulative time spent on CPU. This procedure can be performed locally, but the procedure defined in Section 3.2.1 requires that the locally collected data is augmented with the data collected from peer nodes.

Visibility Class	Description	Systems
Local Insights	Insights can be generated <i>fully locally</i> on the hosts themselves. For instance, individual pod metrics can be used to implement resource management, overload detection, etc.	Sieve [49], DAGOR [56]
Group Data Sharing	Nodes can be grouped into a <i>peer-group</i> based on some static or dynamic property. Certain tools require data sharing among nodes in a peer-group, but insights can still be generated locally. For instance, peer-similarity methods can be used on nodes in a MapReduce cluster to diagnose faults.	Kahuna [48], Ganesha [38]
Global and Local Insights	Insight generation can be split locally and globally. Some components of these techniques can be performed locally, such individual host monitoring, but other parts must be performed globally with data received from all nodes, such as aggregating local insights from each node for root cause analysis.	Seer [22], 007 [11], PARTIES [14]

Table 3.1: Survey of Existing Observability Tools

- **Global and Local Insights:** While some components of these techniques can be performed locally at the node-level, other parts must be *performed globally with data received from all nodes*. For instance, in Seer [22], the deep learning models require centrally collected tracing data across all nodes to predict upcoming QoS violations. Once a faulty node is detected, however, the low-level hard monitoring for why performance is degraded can be performed *locally*. This class of queries motivates the need for some global data collection pipeline and shows that not all insights can be generated locally.

For example, an SRE could decide to autoscale with Sieve based on the P90 memory of all services in a cluster (as before) but then mandates that only the top n overloaded services across the cluster in a period are granted additional resources. This requires local insights to be globally aggregated when selecting the n services to scale out.

Visibility Class	Plane	Requirements
Base	Control	Specifying configurations and custom algorithms for queries and insight generation
	Data	Registering, configuring, and dynamically updating metric collection and insight generation
Local Insights	Control	Accepting and processing local insights from data plane nodes
	Data	Efficiently representing and analyzing collected data
Group Data Sharing	Control	Creating user-specified peer-groups by deriving a key from each node and grouping by this key
	Data	Adding new nodes to a peer-group Streaming new data in a multicast to peer nodes
Global and Local Insights	Control	Aggregating local insights from data plane nodes
	Data	Exporting local insights to a central entity

Table 3.2: Observatory Control and Data Plane Requirements

3.2.5 Control and Data Plane Requirements

For each visibility class, we can now define the *control and data plane requirements*, a set of primitives and workflows, needed for realizing Observatory. To support queries of each visibility class, we require additional workflows and interface specifications per class, starting with a set of base requirements needed for all queries. These requirements are listed in Table 3.2.

3.3 Observatory and eBPF

Whether insights are generated locally or globally, Observatory must fundamentally collect metric data in the data plane. In this section, we discuss the motivation for using eBPF in our data plane in terms of efficiency and flexibility.

3.3.1 Efficiency

Many existing observability solutions scrape Linux kernel metrics found in files under the `/proc` directory. Content in these files contains valuable system metrics, but this form of metric collection does not scale well to larger deployments where hundreds of files need to be read and manually

parsed to export system metrics, incurring extra overhead in user-space.

To address efficiency, eBPF has specialized data structures which can be leveraged for observability (*e.g.*, arrays or maps of arrays) with a low footprint. Since eBPF programs have to run in the kernel, data structures and programs must be verifiable within strict performance bounds [33].

3.3.2 Flexibility

Given the set of all supported metrics in an observability frameworks, we can either collect *all* or a *subset* of metrics. Furthermore, due to the dynamic nature of on-demand offloading, new observability sources will need to frequently be created and deployed. We want to design Observatory to dynamically update the set of metrics being collected. Finally, while existing system metrics can be found in files under the `/proc` directory, parsing these files does not provide the ability to define and collect new, use case specific metrics. In certain scenarios, some of the information available in `/proc` may not even be useful in assessing the state of the system [23].

To address flexibility, eBPF allows us to write custom programs for collecting metrics. These user-defined programs are key to realizing dynamic metric configuration and custom, use case specific metrics.

Chapter 4

Design

We now discuss the design for each core component of Observatory, based on the motivation in Chapter 3, to realize our vision for a dynamic, decentralized observability framework. Observatory supports online, dynamic metric configuration with flexible insight generation: once-centralized analyses can be pushed down to individual nodes to generate *local insights* alongside data collection. Observatory also supports *global insights*, which may require aggregating local insights at the control plane.

Based on our previously discussed motivations, our key challenges in realizing our vision for Observatory are developing:

1. *A scalable and flexible analysis framework that supports all classes of visibility queries:* To create scalable control and data planes, we design and evaluate specialized data structures for data collection. To create a flexible analysis framework, we design a query language as a layer of abstraction on top of our eBPF probes to dynamically configure queries.
2. *Techniques for pushing down insight generation to the data plane:* Our challenges here are designing workflows for turning techniques and algorithms originally meant to operate on centralized data into those that can operate on decentralized, local data. Our **Groupby** primitives, generic *Observatory Data Format*, and overall primitives and workflows for coordination enable us to realize techniques for pushing down computation to individual nodes.

4.1 Design Overview

Observatory requires two types of entities to be deployed: a **nodectl** and a collection of **nodelets**. The **nodelets** form the data plane, and each application operates as a sidecar on a host, generating

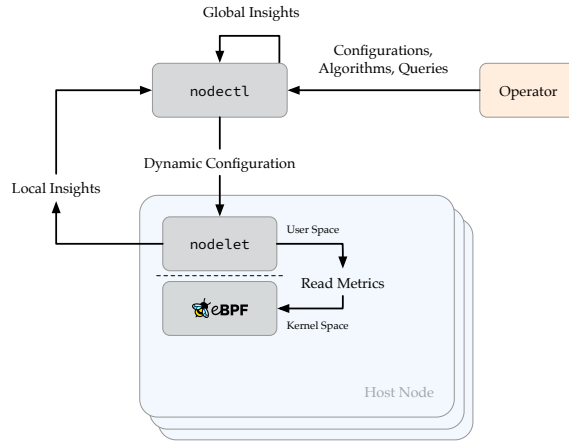


Figure 4.1: Observatory Architecture

telemetry data and possibly running offloaded procedures. A logically centralized **nodectl** serves as the control plane and is responsible for interacting with operators, orchestrating the **nodelet** agents on each data plane node, and possibly running centralized procedures. Operators interact with the **nodectl** by executing queries, writing custom algorithms for generating insights, and specifying configurations, *i.e.* the maximum amount of data to cache at each **nodelet**.

This architecture is shown in Figure 4.1. To achieve our desired division of responsibilities, the **nodelet** awaits instructions from the **nodectl** on what services to monitor, what metrics to activate, how to analyze the resulting data, and where and when to export collected metrics.

4.2 Data Collection Pipeline

Observatory is designed to provide precise control over data collection by collecting metrics *per-pod* in Kubernetes deployments. By focusing on system metrics over application-level metrics, logs, and distributed traces, Observatory does not require instrumenting the target application.

To scale, Observatory minimizes overheads in terms of memory and network when collecting, storing, and exporting metrics. Our data plane implements dynamic metric configuration to only collect metrics specified by the control plane and introduces specialized histogram-based data structures for efficient data representation. This allows us to avoid sampling and aggregating by reducing both the number of metrics to be collected and the volume of data.

4.2.1 Pod Discovery and Dynamic Configuration

Observatory is designed to be Kubernetes native, and our workflows are in turn designed to deploy alongside a Kubernetes deployment. One such workflow is *pod discovery*, or the mapping of which pods (running specific services) are deployed onto which nodes. Since operators can specify which services are to be monitored, what metrics are enabled per-service, and any additional configurations required per-metric, pod discovery is crucial to dynamically configuring our data plane.

For pod discovery, we watch Kubernetes events at the `nodectl` and (de)register containers, metrics, and analyses on the corresponding `nodelet` based on lifecycle events. This workflow forms the control plane portion of dynamic configuration, while the data plane portion of dynamic configuration is implemented with eBPF and discussed in Section 4.3.

4.2.2 Histogram Data Structures

As motivated in Section 3.1, we store our collected metrics in specialized data structures to prevent the need for data sampling. This data structure is implemented as a histogram for discrete data with equal width bins, with a sample configuration in Figure 4.2. Instead of collecting and storing raw metric data, *e.g.*, individual run queue latency times, we instead store *the counts of observed data for each bin*. For instance, if we observe a run queue latency time of *90ms*, we first apply a user-defined bin function on *90ms*, and then increment the corresponding bin by one. In Observatory, we use an exponential function to define our bins, and as we show in Section 7.2, exponentially increasing bins are optimal in terms of overhead and accuracy when compared to large, linear bin histograms. An example of a histogram collected with exponential bins is shown in Figure 4.3 where the metric collected is the TCP round trip time in nanoseconds.

```
typedef struct metric
{
    unsigned int slots[MAX_SLOTS];
    uint64_t ts;
} metric_t;
```

Figure 4.2: Histogram Data Structure Definition

Our histograms are constructed in the kernel-space eBPF programs. We use BPF map structures to store a separate histogram for each monitored pod and use atomic `__sync_fetch_and_add` operations to increment the corresponding bin. The user-space program stores a *timeseries of cumulative histograms*, allowing users to take the difference between any two histograms to understand the change in behavior over an arbitrary timeframe. A circular buffer is used to store this timeseries data to cap memory usage and automatically discard stale data.

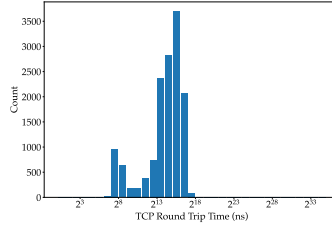


Figure 4.3: Example Histogram in Observatory

4.2.3 Data Streaming, Ingestion, and Processing

Observatory natively supports three *collector schemes*, methods by which the data plane exports data to other entities. These collector schemes are illustrated in Figure 4.4, and each scheme corresponds to a query visibility level described in Section 3.2.4. We show in this section how streaming, ingestion, and processing work for each of these three collector schemes.

Local Streaming

- *Exporting & Ingestion.* No data is exported by default. Observatory will only relay *insights* to the control plane. Insights are use case specific but can take the form of *alerts* when certain conditions are met (*e.g.*, there being an outlier pod) or *requests* for the control plane to take some action (*e.g.*, request to scale out a service after a local decision is made).
- *Processing.* Processing happens locally at each **nodelet**. Analysis procedures are periodically called on the local data.

Global Streaming:

- *Exporting & Ingestion.* All nodes export to a single collector in the control plane. This collector receives a timeseries for each metric in a round buffer and indexes their timestamps by proximity. Observatory can either export data directly to the **nodectl** or use existing data collection pipelines (*e.g.* Prometheus [41]).
- *Processing.* Processing happens globally at the **nodectl**. Analysis procedures are periodically called on the synced data.

Peer-Group Streaming:

- *Exporting & Ingestion.* Each **nodelet** engages in a *multicast* by streaming new data to a subset of nodes in the cluster. A naive multicast would send all data collected on each node

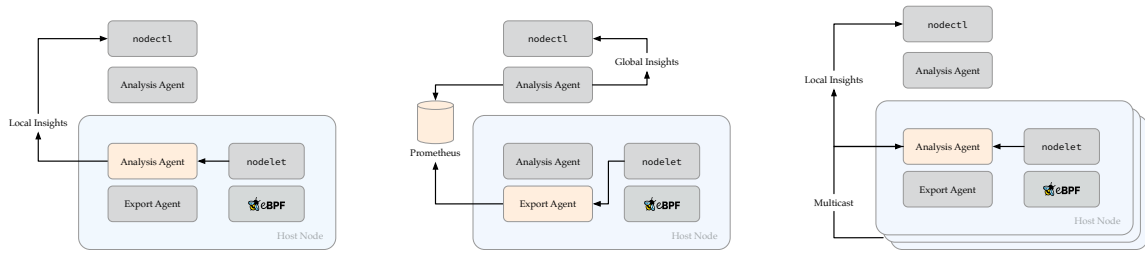


Figure 4.4: Collector Schemes in Observatory

to all other nodes, incurring overhead proportional to the number of nodes in the cluster. To optimize our multicast, we use the notion of *peer-groups* of nodes. These groups are defined by the control plane. Each node then elects a *leader node* for each *peer-group* and only sends data to that leader node.

- *Processing.* Processing happens locally at each `nodelet` in the same manner as the *local streaming* collector scheme.

4.3 eBPF in the Data Plane

As motivated in Section 3.3, Observatory uses eBPF in the data plane to collect metric data and achieve dynamic metric configuration. Developers can add their own eBPF code for custom metrics that will be packaged and deployed with the `nodelet`. We discuss in this section how we design our data plane with eBPF.

- **Custom eBPF metrics:** Custom metrics can be collected by writing new eBPF programs. We define a *kernel-space metric handler* to be a kernel-space program that attaches to some tracepoint. These are user-defined and reflect operating system level constructs that allow Observatory to programmatically calculate system metrics. For instance, a kernel-space metric handler can attach to the kernel scheduler to expose system metrics related to CPU time. Some examples of eBPF metrics in Observatory can be found in Table 4.1.
- **Efficient interface between user and kernel space:** To read metrics and dynamically modify metric collection, Observatory leverages BPF maps, a generic storage mechanism for sharing data between kernel-space and user-space. During the kernel-space metric handlers, derived data or data found kernel task structures are copied into buffers that are stored in the BPF maps. We define a *user-space metric handler* to be a user-space program that reads out metric data from the BPF map for further processing, annotating, and exporting. Each

metric has a corresponding user-space metric handler and kernel-space metric handler, which together form the code necessary to generate telemetry data.

- **Dynamic probe control:** Since containers isolate their processes within a unique PID namespace [37], we can monitor all processes belonging to given pod by using the PID namespace of its container. On each `nodelet`, we store metadata about which pods are actively being monitored and which metrics are being monitored for each pod using a single BPF map. Updates to this map from the user-space (when registering a new pod or metric dictated by the `nodectl`) can be seen in the kernel-space (specifically, during the kernel-space metric handler) when deciding whether to collect metric data for a given process.

Metric	Tracepoint	Description
ON_CPU	<code>sched_switch</code>	Measures the cumulative time spent by each process on the CPU
RUNQ_LAT	<code>sched_switch</code>	Measures the time spent waiting on the kernel run queue before switching back onto the CPU
MALLOC	<code>malloc</code>	Measures the number of bytes requested per process by <code>malloc</code> and <code>realloc</code> calls
TCP_SEND_FLOW	<code>tcp_sendmsg</code>	Measures the number of bytes sent per process over IP
TCP_RCV_FLOW	<code>tcp_cleanup_rbuf</code>	Measures the number of bytes received per process over IP
TCP_RTT	<code>tcp_rcv_established</code>	Measures the round trip time per process for packets sent over TCP

Table 4.1: eBPF Metrics in Observatory

Chapter 5

Observatory Queries

In this section, we define Observatory’s Query Language. This provides a layer of abstraction over the design and primitives in our framework and provides a user-level query language to interact with Observatory. The constructs for the query language can be found in Table 5.1, and the full syntax of the language can be found in Figure 5.1.

5.1 Query Language Overview

Our goal with designing a query language is to abstract away implementation details associated with a query from the SRE interacting with Observatory. These details include **what data** to collect from **which nodes**, how to **generate insights** from the data, and whether computation can be **pushed down**. Our query language draws inspiration from SQL, an existing query language used by operators and data scientists to analyze data stored in databases (*e.g.* MySQL, Postgres). At a high level, an SRE specifies the *data to collect* and the *insights to generate*, and Observatory handles the dynamic configuration needed to fulfil the query.

Construct	Description
<code>Insight(<Probes>)</code>	Specifies the insight to query
<code>.From(Selection)</code>	Defines to which entities the query applies
<code>.Interval(Time)</code>	Specifies the interval between runs
<code>.Apply(Fn, Mode, ListOfArgs)</code>	Apply a function to the results

Table 5.1: Observatory Query Language Constructs

```

<Query> ::= Insight <Probes>
         .From <Selection>
         .Interval Time
         .Apply <Fn>, <Mode> <ListOfArgs>
<Probes> ::= <Probes>, <Probe>
           | <Probe>
<Probe> ::= ON_CPU
           | RUNQ_LAT
           | ...
           | TCP_RTT
<Selection> ::= Pod
             | Service
             | Node
<Mode> ::= Local
         | Global
         | PeerGroup
<Fn> ::= Procedure(<ListOfArgs>)
<ListOfArgs> ::= <ListOfArgs>, Arg
              | Arg

```

Figure 5.1: Observatory Query Syntax

5.2 Language Constructs

We now describe the semantics of our query language and how each construct maps to primitives designed in Observatory.

Insight(<Probes>): The insight is the root construct of an Observatory query where we specify what data to collect, taking the form of a list of eBPF probes. Dynamic metric configuration in Observatory will register these probes on the corresponding data plane nodes.

From: The **From** statement will only look at a subset of entities. A selection is based on specific **pods**, **services**, or **nodes** in the cluster, and including this construct will limit data collection and insight generation to only specified entity.

Interval: The **Interval** construct allows the user to specify the granularity of the querying. As we saw in Figure 3.1, different queries operate at different timescales, and more frequent data collection may be needed for the system to be performant. Specifying a smaller **Interval** will increase the rate at which data is collected and insights are generated.

Apply: Finally, **Apply** allows a user to invoke blackbox functions to the data resulting from the Observatory query. An **Apply** takes a **mode** as input, allowing the user to specify *where the analysis should be performed*. This directly corresponds to the three classes of visibility described in Section 3.2.4. For example, **Apply("Autoscale", Local)** will locally perform autoscaling at each data plane node. Users can write their own functions that operate on metric data to fully customize Observatory.

<code>Insight(RUNQ_LAT)</code>	<code>Insight(ON_CPU)</code>	<code>Insight(RUNQ_LAT)</code>
<code>.From("cartservice",*,*)</code>	<code>.From(*,*,*)</code>	<code>.From("cartservice",*,*)</code>
<code>.Interval (1ms)</code>	<code>.Interval(1ms)</code>	<code>.Interval (1ms)</code>
<code>.Apply("Autoscale", Local)</code>	<code>.Apply("Outlier", PeerGroup)</code>	<code>.Apply("Autoscale", Local)</code>
		<code>.Apply("RankedAutoscale", Global)</code>
(a) Local Sieve	(b) Kahuna	(c) Global Sieve

Figure 5.2: Example Queries in Observatory

Apply Chaining. A powerful feature that enables the full flexibility and customizability of Observatory queries is *apply chaining*, where a user can specify multiple `Apply` statements in a query that operate one after another. Apply chaining can be used to implement the global and local insights class of queries defined in Section 3.2.4. For instance, Figure 5.2c corresponds to local autoscaling decisions with Sieve followed by globally ranking overloaded services before scaling out¹.

Data Format. Observatory’s *data format*, shown in Figure 5.3, creates a structured and standardized way to transport both data and insights between core components. The `Histogram` message allows efficient transportation of the collected data, *i.e.*, in the same format as it is collected with our specialized histogram data structures. We show the low network overhead associated with transmitting exponential bin histograms using this data format in Figure 7.5. The `Insight` message is set in an `Apply` construct and allows the user to store arbitrary data associated with the insight in the `insightDict` field. For instance, a key-value pair in an outlier detection insight could take the form of

```
<"outlier", Value("cartservice")>
```

With apply chaining, `Insight` messages are passed from one user-defined function to another, and a downstream function can expect certain key-value pairs in the `insightDict` field. For instance, in the query shown in Figure 5.2c, `RankedAutoscale` can require insights from upstream functions to have the following key-value pair in `insightDict`:

```
<"service", Value("frontend")>
```

Thus, we show how our data format acts as the glue when transporting insights and data between the components of Observatory, enabling our vision for flexibility and customizability.

¹The "Global and Local Insights" example described in Section 3.2.4

```

message Histogram
{
    uint64          ts = 0;
    map<int64, int64> slots = 1;
}

message Value {
    int64          int_t = 0;
    bool          bool_t = 1;
    string        string_t = 2;
    ...
}

message Insight
{
    uint64          ts = 0;
    uint64          insightID = 1;
    map<string, Value> insightDict = 2; // Data associated with insight
    Histogram      data = 3;
}

```

Figure 5.3: Observatory Data Format

Chapter 6

Prototype & Setup

6.1 Prototype

The prototype of Observatory is built as a Golang application (1.5K LoC) for the `nodectl` in the control plane and a C/C++ application (3K LoC) for the `nodelet` in the data plane. Analyses to generate both local and global insights are written in Python (700 LoC). To develop the BPF components of Observatory, we use `libbpf`, a userspace library and toolkit for eBPF programs. We use gRPC [4] for communication between the control plane and data plane.

6.2 Evaluation Setup

All of our experiments are performed on Google Cloud Platform (GCP) Compute Engine with a cluster composed of 1 control plane node and 5 data plane nodes, each with 4 vCPU and 16Gb of memory. We deploy Google Cloud Platform’s microservice Online Boutique [6], a demo application composed of 11 microservices. Locust [5] is used to simulate load by issuing user requests to the microservice application.

6.3 Use Cases

Our goal in this section is to show how we can implement existing observability tools and techniques in Observatory that fall under various visibility classes of queries. While we previously discussed these use cases in Section 3.2 in the context of motivating the design of Observatory, in this section, we will describe our experiment setup for each use case.

6.3.1 Problem Diagnosis with Kahuna

Kahuna [48] is a problem diagnosis approach that relies on *peer-similarity*. Nodes are claimed to exhibit similar behavior without performance problems, whereas nodes that behave significantly different from their peers are likely to be sources of performance problems. Since detecting outliers requires knowledge of all peer nodes, we can implement problem diagnosis with Kahuna to evaluate the *Group Data Sharing* query class of Observatory.

Experiment Setup. The experiment setup for problem detection with Kahuna is illustrated in Figure 6.1. To evaluate the implementation and effectiveness of Kahuna in Observatory, we first run the microservice application with baseline load in our cluster and horizontally scale *one service* to three worker pods. During the experiment, we inject a single, targeted fault aimed to disrupt the performance of *one* of the replicated pods. Specifically, we CPU throttle the selected pod and evaluate Observatory’s ability to detect and localize this performance problem.

Since performing outlier detection requires knowledge of the state on other nodes, we use the multicast collector scheme defined in Section 4.2.3. We group by service to form the peer-group and choose a single leader to receive data from all nodes where an pod of the monitored service is deployed, and this leader performs the Kahuna outlier detection algorithm. Specifically, at each timestamp, we generate a vector of transformed metric data for each pod. We then compute the pairwise distance between the vectors of every pair of pods using the square-root of the Jensen-Shannon divergence [17]. If the distance from some pod’s vector is further away from a majority of vectors of other pods, we declare that pod to be an outlier.

We track the history of each detection result and compare the first time-to-detection (TTD) in two experiment scenarios:

1. **Local Insights.** We use the fully distributed Observatory configuration with local comparisons on the data plane for outlier detection. After data collection on each node, the multicast exporting scheme is used. At some fixed interval (*i.e.* 2 seconds), each node streams new data to the peer-group leader, and the leader performs outlier detection.
2. **Global Insights.** We perform metric collection as before, but instead of the multicast exporting scheme, we register new data with Prometheus. At the `nodectl`, we periodically pull data from Prometheus at a fixed interval (*i.e.* 2 seconds) and perform outlier detection.

6.3.2 Autoscaling with Sieve

Sieve [49] is a platform to generate actionable insights from a distributed system. Sieve itself was shown to be effective in supporting a wide range of workflows, and our goal in this section is to implement one of these workflows in Observatory: orchestration of autoscaling.

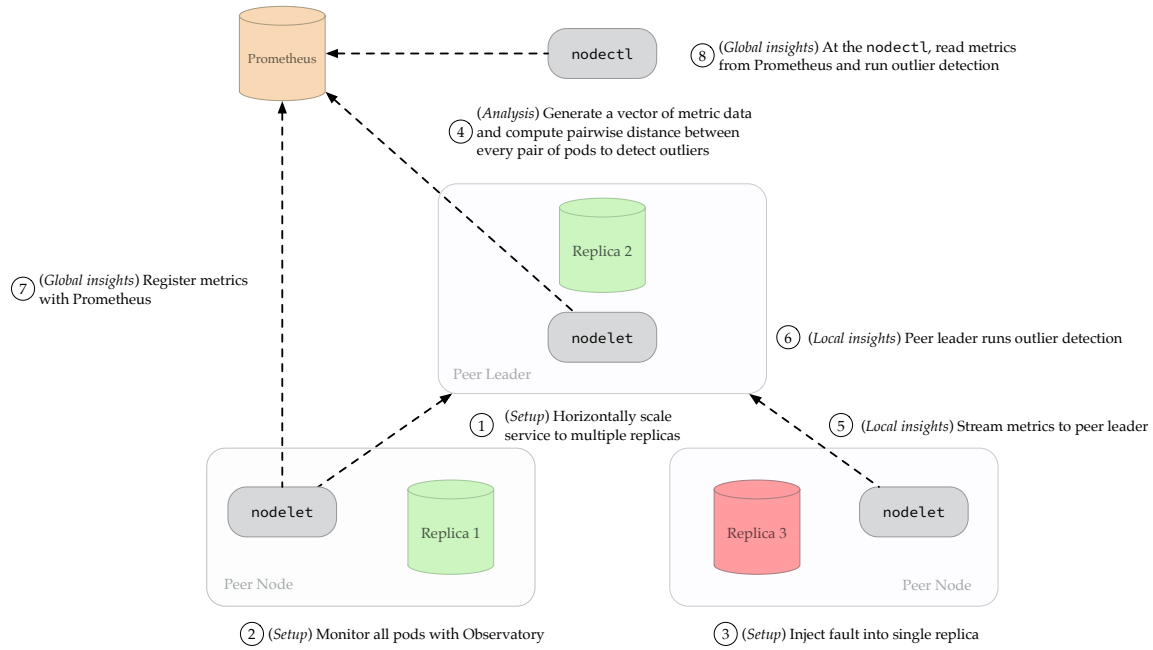


Figure 6.1: Kahuna Experiment Setup

Sieve’s autoscaling engine first uses *k-Shape* [39] clustering to reduce the dimensionality of the provided metrics and chooses a representative metric for each cluster. Then, using a call graph for the microservice, Sieve performs pairwise comparisons using Granger Causality tests [49] between the representative metrics of one service and all neighboring services. They pick the metric m that appears in the most relations and use this metric for autoscaling decisions. Once a metric m is selected, all further insights can be performed locally, allowing us to evaluate the *Local Insights* query class of Observatory.

Experiment Setup. The experiment setup for autoscaling with Sieve is illustrated in Figure 6.2. To evaluate the implementation and effectiveness of Sieve in Observatory, we first run the microservice application with baseline load in our cluster. During the experiment, we surge the load by increasing the number of user requests made to the application by a factor of 10. We evaluate Observatory’s ability to detect this surge in load and scale out the overloaded pod.

When detecting the surge in load, we assume each node *only* requires the knowledge of its performance, leaving the work of cross-node collaboration for autoscaling as future work. Specifically, at some fixed interval (*i.e.* 0.5 seconds), we use our timeseries of histograms of metric data to create a sliding window of performance. Given the metric data seen in the past n seconds, we calculate the P90 threshold of this histogram and compare this to a user-specified target to determine whether

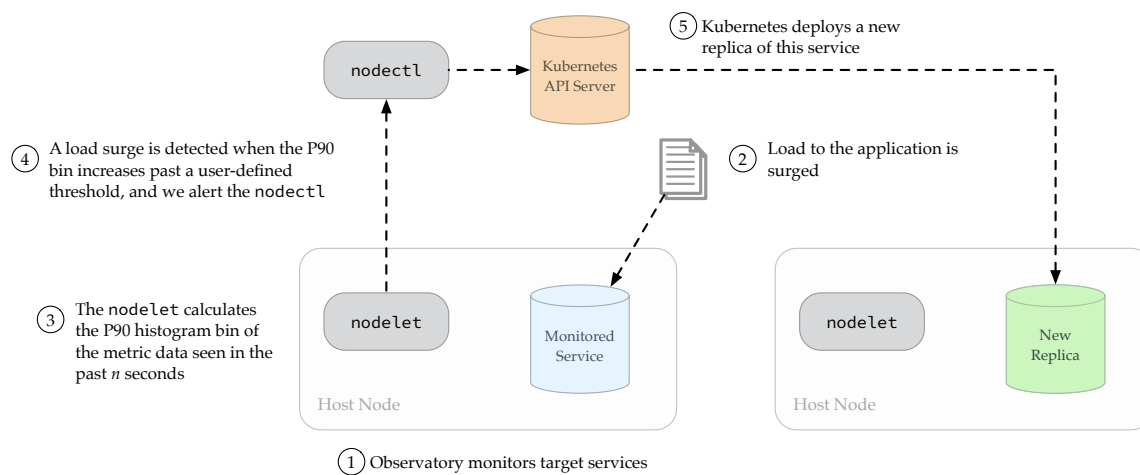


Figure 6.2: Sieve Experiment Setup

to scale out. If a load surge is detected, the P90 threshold of the distribution will increase past the user-defined target, and the data plane will relay this insight to the control plane. The control plane will then request Kubernetes to increase the number of replicas for the overloaded service.

Chapter 7

Evaluation

Our evaluation addresses the following questions:

1. How general is Observatory, and how effective is local insight generation?
2. What are the performance gains of our eBPF-based data structures?
3. How do the control and data plane components scale?

7.1 Use Cases

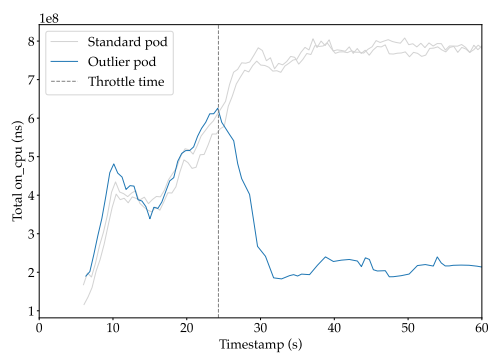
In this section, we demonstrate how Observatory is able to successfully implement the use cases motivated in Section 3.2 and how effective Observatory is in creating high-quality local insights.

7.1.1 Kahuna Problem Detection Results

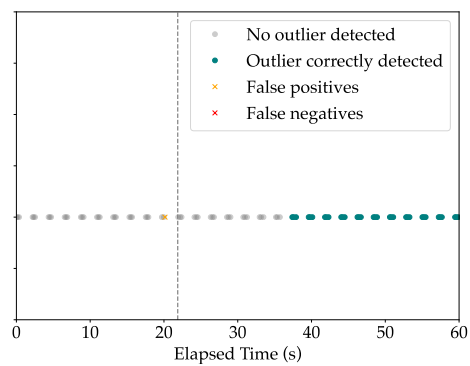
The results for our first use case, problem detection with Kahuna, can be found in Figure 7.1. In this experiment, we evaluate the ability for Observatory to push down computation and create high-quality local insights compared to generating insights globally.

In Figure 7.1a, we show the raw data that is collected at the data plane after the multicast between peer nodes. After the CPU is throttled, we observe that the cumulative on-CPU time drops by over 60% for the outlier pod.

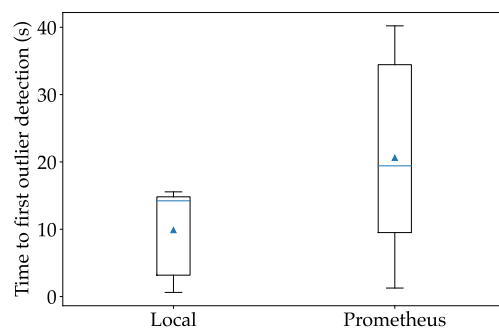
Using this data, our local Kahuna outlier detection procedures run periodically at data plane nodes, and we measure the earliest time to detection of the outlier pod. We illustrate one trial of this experiment in Figure 7.1b. In this trial, we see almost no false positives being detected until



(a) On CPU Time per Pod



(b) TTD Local Insights Timeline



(c) Obsevatory vs Prometheus TTD

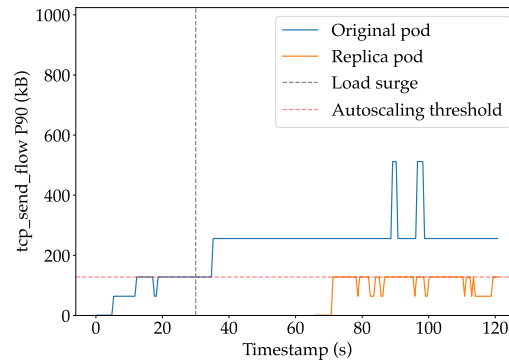
Figure 7.1: Kahuna Problem Detection in Observatory

the CPU is throttled, after which Observatory is able to correctly identify the outlier pod with no false positives.

To evaluate the quality of local insights, we compare the time to detection resulting from local insights with that from global insights, as shown in Figure 7.1c. We see that time to detection increases when performing outlier detection at the control plane after reading data from Prometheus, from a mean of 9.878 seconds to 20.620 seconds, an increase of 52%. We also observe a greater variance in detection times when performing outlier detection at the control plane. While detection accuracy is unaffected, the higher variance and greater time to detection show that the *quality* of insights is degraded with global insights when compared to local insights.

7.1.2 Sieve Autoscaling Results

The results for our second use case, orchestration of autoscaling with Sieve, can be found in Figure 7.2. In this experiment, we further evaluate the ability to push down computation with fully local insights.



(a) TCP kB Received P90

Figure 7.2: Sieve Autoscaling in Observatory

In Figure 7.2a, we show the P90 bytes sent for the monitored service before and after the surge in load. Since Observatory uses exponential bins, the surge at 30 seconds results in an order of magnitude increase in P90 bytes sent, from 2^7 bytes to 2^9 bytes.

We then observe that using the statically determined autoscaling threshold, the original pod picked up this surged load and alerted the control plane at 34.72 seconds, only 4.72 seconds after the initial surge, and Kubernetes deployed a new replica pod to service requests. This further demonstrates Observatory’s effectiveness in generating high-quality, local insights.

7.2 eBPF-based Data Structures

Our goal in this section is to evaluate the performance characteristics of our eBPF-based data structures. We perform a series of micro-benchmarking experiments and compare the overhead and accuracy of three designs for our custom data structures for metric collection: large, linear bin histograms, exponential bin histograms, and fixed-sized arrays.

In this experiment, we construct the distribution of run queue latency times. All three data structure designs are backed by eBPF maps. Conceptually, the histogram data structure stores the *counts* of observed metric data for each bin, whereas array-backed data structure stores the *raw observed metric data*. Data collection with the array-backed data structure uses the following methodology:

1. At the data plane, we record observed run queue latencies in a buffer, sampling the data written to the buffer. In user-space at some fixed interval, we read and clear the buffer to export the observed data. While the fixed-size array prevents excess memory usage, the buffer may reach capacity between intervals. These raw observed metric data are exported to the control plane.
2. At the control plane, we reconstruct the distribution using this sampled data.

To evaluate the accuracy of the data collected for each data structure design, we compare the constructed distributions given the same backing data. Specifically, we measure the relative error of the P90 histogram bin at each point in our timeseries data. Before calculating the relative error, we normalize the P90 histogram bin to the original scale to account for the different number of bins in the histograms (*e.g.*, a bin index of 7 in the exponential-bin histograms is normalized to 2^7). Our *ground truth* distribution for comparison is the large, linear bin histogram with 4096 slots. For comparison, the exponential bin histogram is constructed with 36 slots.

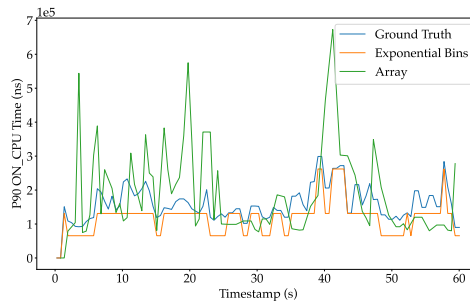


Figure 7.3: Comparison of P90 on-CPU Time

Figure 7.3 shows the P90 on-CPU time for each data structure design after applying this normalization, and Table 7.1 shows the mean relative error. We see the mean relative error is upwards of 30% greater when using array-backed histograms.

Percentile	Exponential Histogram Error (%)	Array-Backed Histogram Error (%)
0.50	21.364	40.542
0.90	26.346	55.932
0.95	28.125	79.029
0.99	28.114	62.668

Table 7.1: Mean Relative Error of Distributions

The overhead in data collection for each data structure design is shown in Figure 7.5. We see significantly greater memory and network overhead when collecting and transporting large, linear bin histograms, i.e., MB/s of data received at the control plane dropped from an average of 2.06×10^{-3} with the linear bin histograms to an average of 3.20×10^{-4} with exponential bin histograms (an 84% decrease). Similarly, memory usage at the data plane dropped from an average of 16 MB to an average of 13 MB (a 19% decrease).

7.3 Probe Latency

To assess the overhead associated with the eBPF probes for metric collection, we look at both time spent in kernel-space and overall added latency to requests.

7.3.1 Critical Path Latency

Our *critical path* is defined as the eBPF tracepoint code executed in the kernel. To measure this latency, we vary the number of metrics monitored and track the time spent in kernel-space across all tracepoints. A distribution of these critical path times is shown in Figure 7.4a, and we can see that as the number of metrics increase, the time spent *at any given tracepoint* stays the same.

7.3.2 Request Latency

To evaluate the additional latency in user requests due to monitoring, we vary the number of metrics monitored and track median response times to the monitored service, shown in Figure 7.4b. We see that at the 95% confidence interval, the median response times is the same as the number of metrics increase, showing no significant latency to the end-user request from Observatory.

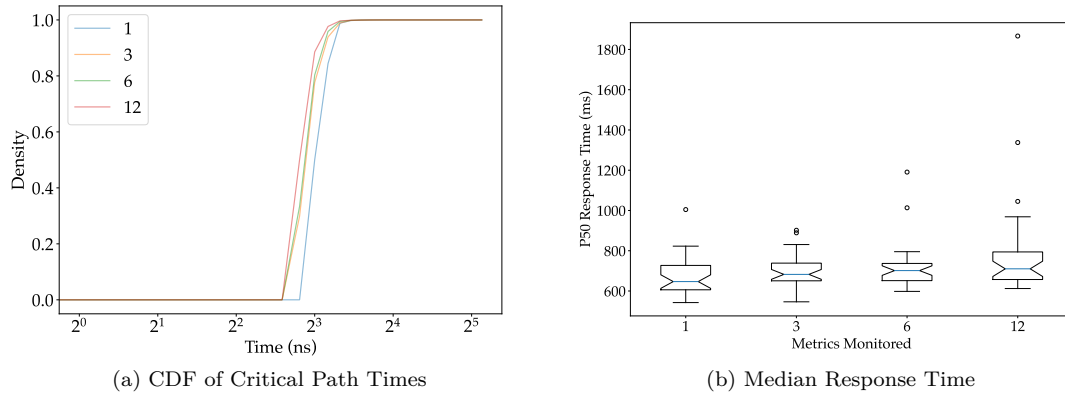
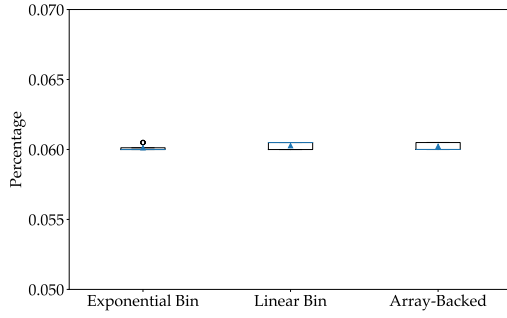


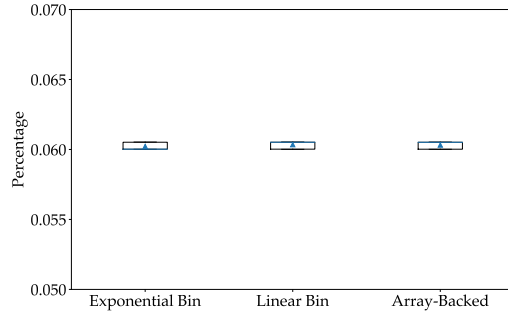
Figure 7.4: Probe Latency

7.4 Control and Data Plane Scalability

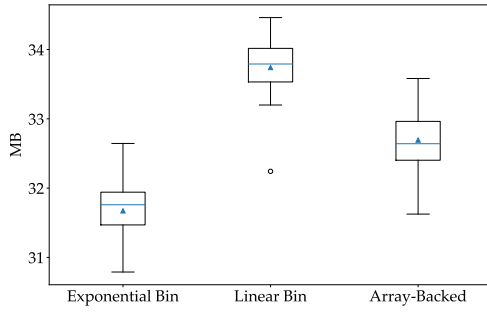
To show how Observatory scales with the size of the set of metrics to collect, we vary the number of eBPF metrics collected under a fixed workload and measure overhead in terms of CPU, memory, and network at the both the control plane and data plane. CPU overhead is calculated as the difference in cycle count at the start and end of the experiment, memory overhead is measured as the maximum resident set size (RSS) used by the application, and network overhead is measured as the number of bytes sent and received by the application. In the case of memory and network overheads, the application is the `nodectl` at the control plane and the `nodelet` at the data plane. These results are shown in Figure 7.6. CPU utilization of both the `nodectl` and `nodelet` remain constant at 0.06%. As expected, we see a linear increase in the memory (*MB* allocated) and network (*MB/s* received and transmitted) as the number of metrics increase.



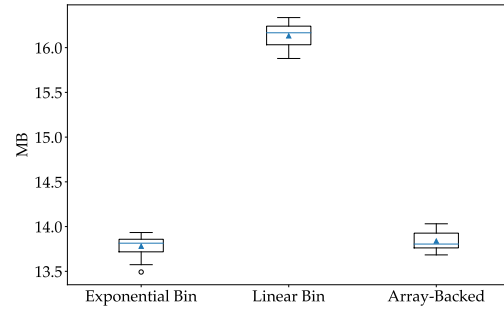
(a) CPU Control Plane



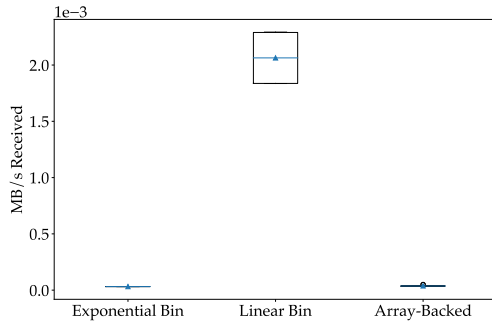
(b) CPU Data Plane



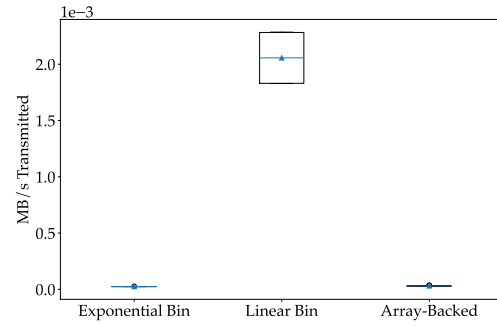
(c) Memory Control Plane



(d) Memory Data Plane



(e) Network Control Plane (Received)



(f) Network Data Plane (Transmitted)

Figure 7.5: Data Structure Microbenchmarking Results

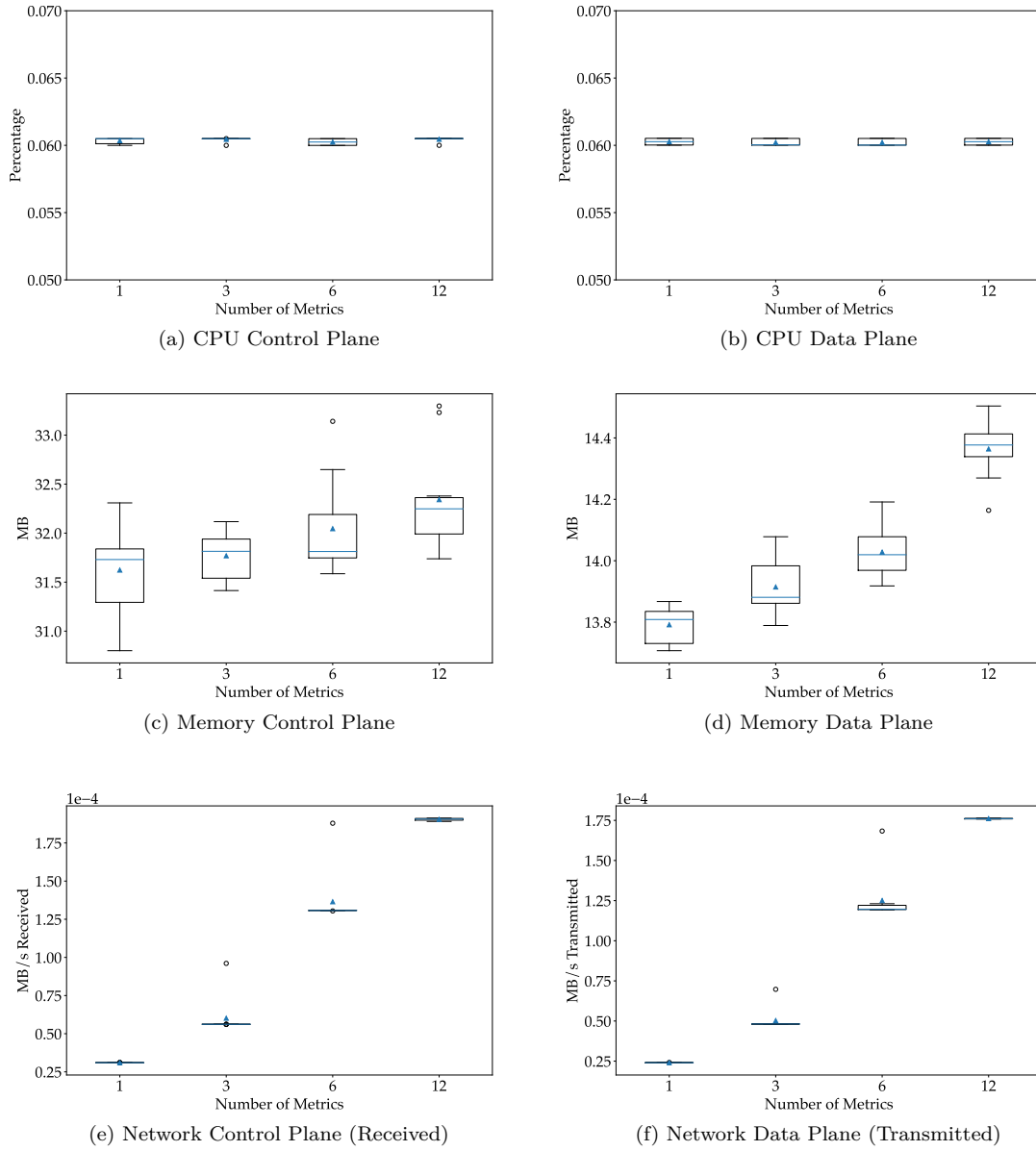


Figure 7.6: Overhead with Increasing Metrics Monitored

Chapter 8

Related Works

Problem Localization in Distributed Applications

Distributed architectures add an extra layer of complexity to diagnosis: unlike monolithic applications, where problems can be analyzed directly at the node-level with existing tools [3, 13], distributed diagnosis needs to first localize the offending component inside an infrastructure (*e.g.*, outliers inside a peer-group) to only then attempt to analyze the problem. To this end, AppDiff [43] records a timeseries of system call distributions and identifies outlier *traces*, or the set of kernel events that process a user request, with conditional probability. PerfScope [16] identifies anomalous sequences of system calls and then maps these sequences back to application-level functions that are likely sources of performance problems. In terms of distributed tracing, Spectroscope [44] tries to identify behavior changes by looking at how distributed traces change in shape and in properties (*e.g.*, trace latency) across time. CRISP [55] performs critical path analysis over distributed traces to identify bottleneck services, displaying heat maps and flame graphs to operators to facilitate performance diagnosis.

Observatory differs from the above techniques foremost in our use of system metrics over log-based analyses or distributed traces. Moreover, Observatory’s push-down mechanisms aim to efficiently distribute computation and generate fast insights. Our framework is designed to support existing problem localization techniques, offloading the required computation when possible.

Non-Traditional Observability Pipelines

We refer to *non-traditional observability* as techniques that stray from the traditional pipeline of pooling all observability data into a centralized location and *only then* processing user queries on the data. For instance, Fay [18] is a platform that efficiently collects, processes, and analyzes execution

traces by dynamically instrumenting functions in both user and kernel space (*i.e.*, processing also happens directly on nodes). Like Observatory, Fay exposes a query language to specify what to trace and how to aggregate trace events from different threads, CPUs, etc, automatically optimizing data collection and analysis based on the user query. Project Trinidad [52] is an industry tool developed at VMware that also uses a zero-instrumentation design for data collection by using eBPF probes to capture network traffic on Kubernetes nodes. Project Trinidad deploys their data collection engine, the Trinidad Sensor, on every Kubernetes worker node. Collected data is processed locally and uploaded to the Trinidad Backend to generate insights, *i.e.*, traffic analysis. Zero [53] collects raw metrics on hosts using one-sided remote direct memory access (RDMA) operations to achieve low CPU overhead at their data plane nodes, which “passively” export metrics by exposing their memory addresses to reads from the controller.

While the mentioned systems showcase specific instances of processing being done early-on in the observability pipeline (and the benefits of doing so), Observatory, through our primitives and query language, aims to provide a framework for systematically implementing early-processing into arbitrary observability pipelines.

Dynamic Instrumentation

This class contains works which, similarly to Observatory, can dynamically reconfigure observability code, avoiding the overhead of manual re-instrumentation of application code.

ViperProbe [28] is an eBPF-based observability framework that proposes algorithms and techniques for identifying the “critical metrics”, a subset of all metrics that reduces volume of metrics collected while still capturing the performance profile of the application. Based on these metrics, ViperProbe dynamically enables the necessary probes on data plane nodes to centrally collect telemetry data and generate insights. The Variance-driven Automated Instrumentation Framework (VAIF) [50] automatically enables the logs needed to diagnose performance problems by placing additional tracepoints at high-variance locations in the critical paths of requests, searching through the space of possible tracepoints with strategies like binary search. Pivot Tracing [32] is a monitoring framework that utilizes dynamic instrumentation that allows users to specify desired metrics and exposes a query language to filter, join, and group by properties of the cluster.

Chapter 9

Future Work

We describe in this section a few areas of future work with Observatory and decentralized observability frameworks.

Automatic decomposition of queries: Currently, a user of Observatory is required to manually specify subqueries that can take advantage of our push down mechanisms in generating local insights. Future work involves automatically decomposing a user query into subqueries such that each subquery is able to run in the most efficient and decentralized manner possible. Essentially, we would enumerate to our system the basic subqueries that can be performed and allow the runtime system to decompose the user query in an optimal manner.

Budget constraints on queries: Currently, Observatory will always offload computation to local nodes if specified by the user. This assumes that the data plane nodes will always have necessary resources to generate local insights and that consuming those resources is acceptable at all times. Future work with Observatory involves specifying a dynamic *budget* for each query, that is, the maximum amount of resources that should be used at data plane nodes to fulfill the query. Based on available resources at the data plane node, our framework would choose whether or not to offload computation and the minimum intervals at which these queries could run without violating the budget.

Augmenting distributed traces: The work in [8] augments distributed traces with the metric distributions of the different tasks within the trace for visualizing Gantt charts of distributed traces during performance debugging. Observatory could be extended to automatically augment such distributed traces with the necessary metric histograms.

Chapter 10

Conclusion

In this thesis, we present Observatory, a dynamic, decentralized observability framework where queries can be decomposed and computation can be pushed down onto individual nodes alongside data collection. To realize Observatory, we design workflows for pushing down insight generation to the data plane, and to efficiently represent and transport metric data, we design specialized eBPF data structures. We evaluate Observatory against state-of-the-art observability systems by implementing various techniques within our framework and demonstrate how Observatory is able to decompose queries and effectively push down computation to data plane nodes. Overall, our design of Observatory allows operators to write meaningful queries with precise control over metric collection and generate fast, high-quality insights on collected telemetry data.

Bibliography

- [1] Docker: Accelerated, Containerized Application Development. <https://www.docker.com/>.
- [2] eBPF. <https://ebpf.io/>.
- [3] GDB: The GNU Project Debugger. <https://www.sourceware.org/gdb/>.
- [4] gRPC. <https://grpc.io/>.
- [5] Locust. <https://locust.io/>.
- [6] Online boutique. <https://github.com/GoogleCloudPlatform/microservices-demo>.
- [7] P. Alvaro, K. Andrus, C. Sanden, C. Rosenthal, A. Basiri, and L. Hochstein. Automating failure testing research at internet scale. In *Proceedings of the Seventh ACM Symposium on Cloud Computing*, SoCC '16, page 17–28, New York, NY, USA, 2016. Association for Computing Machinery.
- [8] V. Anand, M. Stolet, T. Davidson, I. Beschastnikh, T. Munzner, and J. Mace. Aggregate-driven trace visualizations for performance debugging, 2020.
- [9] D. Ardelean, A. Diwan, and C. Erdman. Performance analysis of cloud applications. In *Proceedings of the 15th USENIX Conference on Networked Systems Design and Implementation*, NSDI'18, page 405–417, USA, 2018. USENIX Association.
- [10] D. Ardelean, A. Diwan, and C. Erdman. Performance analysis of cloud applications. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 405–417, Renton, WA, Apr. 2018. USENIX Association.
- [11] B. Arzani, S. Ciraci, L. Chamon, Y. Zhu, H. H. Liu, J. Padhye, B. T. Loo, and G. Outhred. 007: Democratically finding the cause of packet drops. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 419–435, Renton, WA, Apr. 2018. USENIX Association.
- [12] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14:70–93, 2016.
- [13] B. M. Cantrill, M. W. Shapiro, and A. H. Leventhal. Dynamic instrumentation of production systems. In *Proceedings of the Annual Conference on USENIX Annual Technical Conference*, ATEC '04, page 2, USA, 2004. USENIX Association.
- [14] S. Chen, C. Delimitrou, and J. F. Martínez. Parties: Qos-aware resource partitioning for multiple interactive services. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '19, page 107–120, New York, NY, USA, 2019. Association for Computing Machinery.
- [15] I. Cloud. Kubernetes vs. docker: Why not both?, June 2022.

- [16] D. J. Dean, H. Nguyen, X. Gu, H. Zhang, J. Rhee, N. Arora, and G. Jiang. Perfscope: Practical online server performance bug inference in production cloud computing infrastructures. In *Proceedings of the ACM Symposium on Cloud Computing*, SOCC '14, page 1–13, New York, NY, USA, 2014. Association for Computing Machinery.
- [17] D. Endres and J. Schindelin. A new metric for probability distributions. *IEEE Transactions on Information Theory*, 49(7):1858–1860, 2003.
- [18] U. Erlingsson, M. Peinado, S. Peter, M. Budiu, and G. Mainar-Ruiz. Fay: Extensible distributed tracing from kernels to clusters. *ACM Trans. Comput. Syst.*, 30(4), nov 2012.
- [19] Facebook. Open-sourcing katran, a scalable network load balancer, May 2018.
- [20] R. Fonseca, G. Porter, R. H. Katz, and S. Shenker. X-Trace: A pervasive network tracing framework. In *4th USENIX Symposium on Networked Systems Design & Implementation (NSDI 07)*, Cambridge, MA, Apr. 2007. USENIX Association.
- [21] Y. Gan, M. Liang, S. Dev, D. Lo, and C. Delimitrou. Sage: Practical and scalable ml-driven performance debugging in microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '21, page 135–151, New York, NY, USA, 2021. Association for Computing Machinery.
- [22] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, and C. Delimitrou. Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '19, page 19–33, New York, NY, USA, 2019. Association for Computing Machinery.
- [23] K. Hess. Important linux /proc filesystem files you need to know, 2020.
- [24] IBM. Microservices in the enterprise, 2021, Mar 2021.
- [25] J. Jackson. Debugging microservices: Lessons from google, facebook, lyft, July 2018.
- [26] J. Kreps. Kafka : a distributed messaging system for log processing. 2011.
- [27] G. Leffler. The hidden cost of sampling in observability, May 2021.
- [28] J. Levin and T. A. Benson. Viperprobe: Rethinking microservice observability with ebpf. In *2020 IEEE 9th International Conference on Cloud Networking (CloudNet)*, pages 1–8, 2020.
- [29] K. Lew and S. Narayanan. Lessons from building observability tools at netflix, June 2018.
- [30] J. Lewis and M. Fowler. Microservices. *martinfowler.com*, Mar 2014.
- [31] J. Mace. End-to-End Tracing: Adoption and Use Cases. Survey, Brown University, 2017.
- [32] J. Mace, R. Roelke, and R. Fonseca. Pivot tracing: Dynamic causal monitoring for distributed systems. *ACM Trans. Comput. Syst.*, 35(4), dec 2018.
- [33] A. Maguire. Bpf in depth: The bpf bytecode and the bpf verifier, Jan 2019.
- [34] I. Nagi and G. Kuncoro. *KubeCon + CloudNativeCon North America 2021*.
- [35] Netflix. Introducing atlas: Netflix’s primary telemetry platform, December 2014.
- [36] F. Neves, N. Machado, and J. Pereira. Falcon: A practical log-based analysis tool for distributed systems. In *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 534–541, 2018.
- [37] S. Ovens. Building containers by hand: The pid namespace, 2021.

- [38] X. Pan, J. Tan, S. Kavulya, R. Gandhi, and P. Narasimhan. Ganesha: Black-Box fault diagnosis for MapReduce systems. San Diego, CA, Dec. 2008. USENIX Association.
- [39] J. Paparrizos and L. Gravano. K-shape: Efficient and accurate clustering of time series. *SIGMOD Rec.*, 45(1):69–76, jun 2016.
- [40] A. Parker, D. Spoonhower, J. Mace, B. Sigelman, and R. Isaacs. *Distributed Tracing in Practice: Instrumenting, Analyzing, and Debugging Microservices*. O'Reilly Media, Incorporated, 2020.
- [41] B. Rabenstein and J. Volz. Prometheus: A Next-Generation monitoring system (talk). Dublin, May 2015. USENIX Association.
- [42] A. Reinholds. What is observability?, November 2021.
- [43] J. Rhee, H. Zhang, N. Arora, G. Jiang, and K. Yoshihira. Software system performance debugging with kernel events feature guidance. In *2014 IEEE Network Operations and Management Symposium (NOMS)*, pages 1–5, 2014.
- [44] R. R. Sambasivan, A. X. Zheng, M. De Rosa, E. Krevat, S. Whitman, M. Stroucken, W. Wang, L. Xu, and G. R. Ganger. Diagnosing performance changes by comparing request flows. In *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation*, NSDI'11, page 43–56, USA, 2011. USENIX Association.
- [45] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan, and C. Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Technical report, Google, Inc., 2010.
- [46] C. Sridharan. *Distributed Systems Observability: A Guide to Building Robust Systems*. O'Reilly Media, 2018.
- [47] S. Srivatsan. Observability at scale: Building uber's alerting ecosystem, November 2018.
- [48] J. Tan, X. Pan, E. Marinelli, S. Kavulya, R. Gandhi, and P. Narasimhan. Kahuna: Problem diagnosis for mapreduce-based cloud computing environments. In *2010 IEEE Network Operations and Management Symposium - NOMS 2010*, pages 112–119, 2010.
- [49] J. Thalheim, A. Rodrigues, I. E. Akkus, P. Bhatotia, R. Chen, B. Viswanath, L. Jiao, and C. Fetzer. Sieve: Actionable insights from monitored metrics in distributed systems. In *Proceedings of the 18th ACM/IFIP/USENIX Middleware Conference*, Middleware '17, page 14–27, New York, NY, USA, 2017. Association for Computing Machinery.
- [50] M. Toslali, E. Ates, A. Ellis, Z. Zhang, D. Huye, L. Liu, S. Puterman, A. K. Coskun, and R. R. Sambasivan. Automating instrumentation choices for performance problems in distributed applications with vaif. In *Proceedings of the ACM Symposium on Cloud Computing*, SoCC '21, page 61–75, New York, NY, USA, 2021. Association for Computing Machinery.
- [51] A. Verma, L. Pedrosa, M. R. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, Bordeaux, France, 2015.
- [52] VMware. Project trinidad: Merging security with modern application observability – part 2: Zero instrumentation for full visibility, Nov. 2022.
- [53] Z. Wang, T. Ma, L. Kong, Z. Wen, J. Li, Z. Song, Y. Lu, G. Chen, and W. Cao. Zero overhead monitoring for cloud-native infrastructure using RDMA. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 639–654, Carlsbad, CA, July 2022. USENIX Association.
- [54] D. Xia. How spotify accidentally deleted all its kube clusters with no user impact. KubeCon + CloudNativeCon Europe 2019, 2019.
- [55] Z. Zhang, M. K. Ramanathan, P. Raj, A. Parwal, T. Sherwood, and M. Chabbi. CRISP: Critical path analysis of Large-Scale microservice architectures. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 655–672, Carlsbad, CA, July 2022. USENIX Association.

- [56] H. Zhou, M. Chen, Q. Lin, Y. Wang, X. She, S. Liu, R. Gu, B. C. Ooi, and J. Yang. Overload control for scaling wechat microservices. In *Proceedings of the ACM Symposium on Cloud Computing, SoCC '18*, page 149–161, New York, NY, USA, 2018. Association for Computing Machinery.